

From Requirements to a Working **Product**

An eight-stage delivery lifecycle for a small team building software with AI assistance. Every stage names its owner, its inputs, its deliverables, the point where AI helps, the point where it must not decide, and an exit test somebody can actually run.

THE CLAIM THIS GUIDE MAKES

Speed comes from **small verified increments**, not from generating more code. AI moves the cost of software from writing to **reviewing and proving**. A process that does not move review and evidence to the front will get faster at producing work nobody can trust.

HOW TO USE IT

This is a **working guide, not a gate**. Stages 01–03 and 05 are set up once and revisited when evidence changes them. Stages 04, 06 and 07 repeat for every increment. Stage 08 never stops. Page 11 tells you what to cut when the team is small.

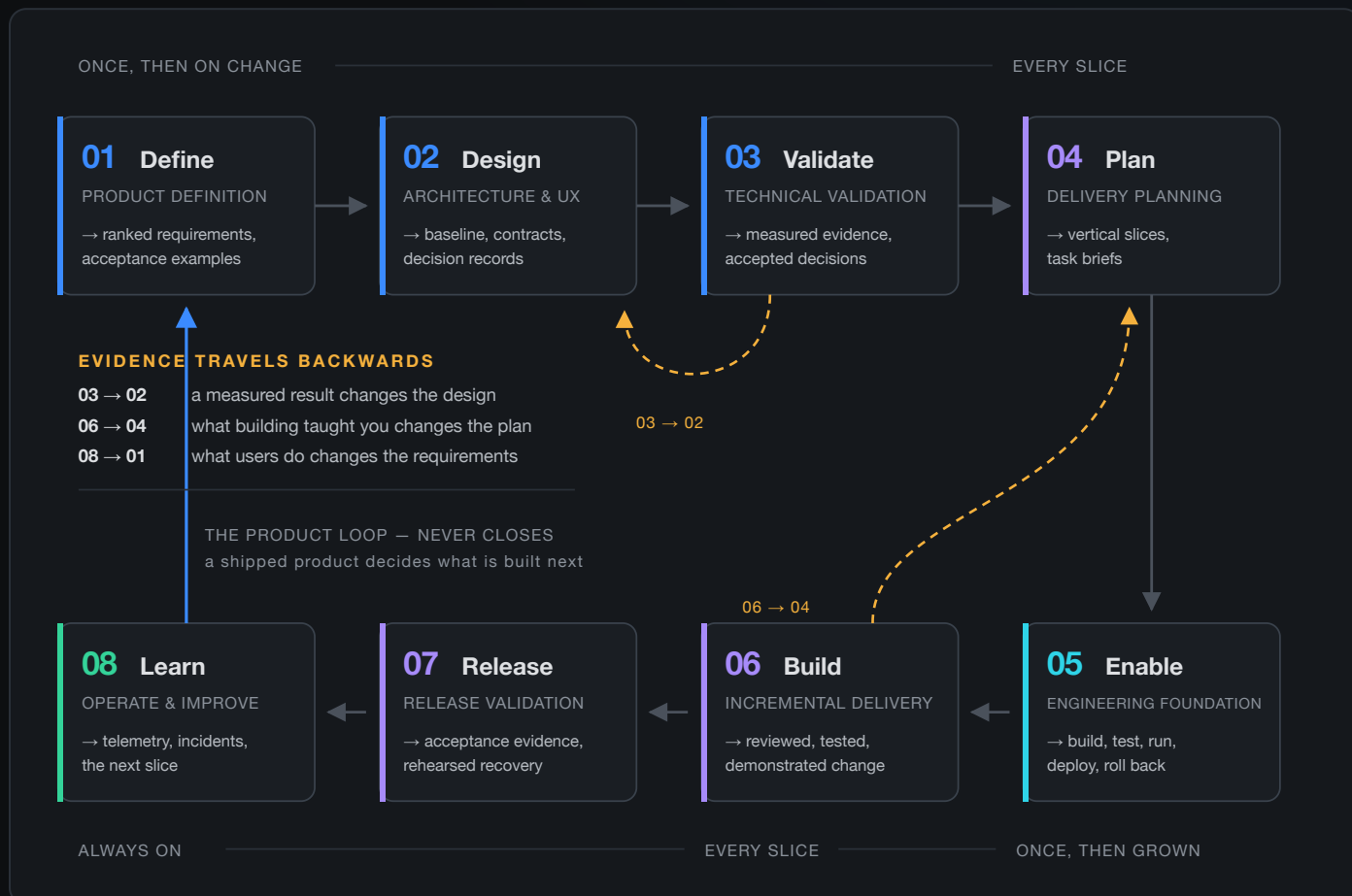
THE EIGHT STAGES

- | | |
|---|---|
| ■ 01 Define · what outcome, for whom | ■ 05 Enable · anyone can build and run it |
| ■ 02 Design · where each responsibility lives | ■ 06 Build · one behaviour, end to end |
| ■ 03 Validate · what could force a redesign | ■ 07 Release · can we support it |
| ■ 04 Plan · the smallest useful increment | ■ 08 Learn · is it useful and improving |

THE SHAPE OF THE WORK

The delivery lifecycle

Eight stages on one circuit. The solid line is the order you first walk it. The dashed lines are the ones that matter more: **evidence travelling backwards**. A lifecycle drawn as a straight line is a lifecycle that has never been corrected by a measurement.



■ SET THE FRAME

01–03. Done once at the start, and again whenever evidence or the market changes an answer.

■ LAY THE GROUND

05. Built once, only as far as the first slice needs, then extended slice by slice.

■ THE DELIVERY LOOP

04, 06, 07. Repeats for every increment. If a lap takes more than two weeks, the slice is too big.

■ ALWAYS ON

08. Runs from the first deployment to the end of the product's life, and feeds every other stage.

HOW TO READ IT

- › The **solid circuit** is the order of a first pass. You walk it once to reach a working product.
- › The **dashed paths** are the corrections. They are normal, not failures. A quarter with no dashed path used is a quarter where nothing was measured.
- › The **blue left leg** is the only loop that never closes. It runs for the life of the product.

AI ACROSS THE CIRCUIT

- › AI assists at every stage. It **owns no stage**.
- › A named person owns each stage's decision and its evidence.
- › Generated work is a **candidate change**. It becomes real when a check has been executed and a human has reviewed the result.

01 DEFINE

02 DESIGN

03 VALID.

04 PLAN

05 ENABLE

06 BUILD

07 RELEASE

08 LEARN

01

SET THE FRAME · ONCE, THEN ON CHANGE

Product definition

What outcome are we building, and for whom? The stage where a wish becomes a testable statement with an owner's name on it.

ACCOUNTABLE OWNER

Product owner, with engineering and user input.

START WITH

User needs, observed behaviour of any system being replaced, constraints, and the goals the business is paying for.

ACTIVITIES

- › Name the users and write their main journeys end to end — **including the unhappy ones**: first use, working through a failure, and the worst thing that can go wrong.
- › Separate four things that are easy to blur: **observed behaviour**, **decided policy**, **technical recommendation**, and **unverified assumption**. Give every requirement a stable ID and one acceptance example.
- › Choose the smallest first release a real user would actually use. Rank the rest. Defer anything whose value or cost you cannot yet state in a sentence.

DELIVERABLES

A versioned requirements baseline, ranked scope, acceptance journeys, measurable service targets, and a short list of open decisions — each with an owner and a date.

WHERE AI HELPS · AND WHERE IT MUST NOT

Drafts and organises requirements, finds contradictions, proposes missing error cases, traces each requirement back to its evidence. **It does not set priority and does not accept a requirement.**

DO NOT START UNTIL

One named person is accountable for scope, and you can name at least one real user or a proxy who speaks for them.

COMPLETION TEST

Pick any requirement in the first release at random. It has an ID, one acceptance example, and a named owner — and someone can state, out loud, how it will be shown to work.

MAKE IT CONCRETE FOR YOUR PRODUCT

Which single journey, if it failed silently, would cost you the most? Write that one first, and make its failure case part of the requirement rather than a later bug report.

COMMON TRAP · Copying every feature of an existing system because it exists. Or promoting an AI suggestion to an approved requirement without recording who decided it.

01 DEFINE

02 DESIGN

03 VALID.

04 PLAN

05 ENABLE

06 BUILD

07 RELEASE

08 LEARN

02

SET THE FRAME · ONCE, THEN ON CHANGE

Architecture and UX

How will the system work, and where does each responsibility belong? Enough design to make the first slice concrete — and no more than that.

ACCOUNTABLE OWNER

Technical lead, with product, UX and operations input.

START WITH

The requirements baseline, the patterns you can already reuse, and the open decisions with the highest cost of being wrong.

ACTIVITIES

- › Draw **three views, no more, to begin with**: a system-context view, a deployable-component view, and one sequence for every journey that crosses a trust boundary. Mark the trust and tenant boundaries on the drawing itself.
- › Assign ownership of every domain and every store: who writes it, who checks permission, and whether its history matters. Write only the API and event contracts the first slice needs.
- › Sketch the key screens against **one** shared design system. Define the loading, empty, offline, expired-session and failure states before anyone builds the happy path.

DELIVERABLES

An architecture baseline, key wireframes, first contracts, a data-ownership matrix, and short decision records (ADRs) each marked **proposed** or **accepted**.

WHERE AI HELPS · AND WHERE IT MUST NOT

Compares options, drafts diagrams and contracts, finds missing transitions, reviews coupling. **Engineers own the trade-off and the accepted decision.** A proposed decision stays visibly proposed until a person accepts it.

DO NOT START UNTIL

Requirements carry IDs and a first release has been chosen. Designing against an unranked wish list produces an unranked architecture.

COMPLETION TEST

A developer traces one request — UI, authentication, authorisation, durable write, event delivery, visible result — by pointing at the drawing, without asking anyone a question.

MAKE IT CONCRETE FOR YOUR PRODUCT

Name the one boundary that carries the most risk: tenant isolation, money, personal data, or safety. Draw that boundary first and put a sequence diagram either side of it.

COMMON TRAP · Designing every future service before proving one user journey. Or assuming frontend module boundaries must match backend service boundaries — they answer different questions.

01 DEFINE

02 DESIGN

03 VALID.

04 PLAN

05 ENABLE

06 BUILD

07 RELEASE

08 LEARN

03

SET THE FRAME · ONCE, THEN ON CHANGE

Technical validation

Which assumptions, if wrong, would force a redesign? This is the cheapest stage in the book and the one teams skip first.

ACCOUNTABLE OWNER

Engineering, together with the owner of the decision being tested.

START WITH

The proposed ADRs, and a ranked list of assumptions that are both **uncertain** and **expensive to change**. Ignore the cheap ones.

ACTIVITIES

- › Write each experiment as a **question with a pass or fail answer, before you build it**. Keep it small enough to finish in days, not weeks.
- › Attack the boundaries: identity and session handling, tenant isolation, permission enforcement, reconnection, and what happens when access is revoked or a dependency disappears while a user is mid-task.
- › Test on the real target devices, browsers and network conditions the product claims to support. A platform's marketing page is not evidence that your use of it works.

DELIVERABLES

Reproducible experiments, measured findings with the date and machine recorded, accepted or revised ADRs, and a risk list with owners.

WHERE AI HELPS · AND WHERE IT MUST NOT

Builds fixtures, small prototypes and failure-injection scripts, and summarises observations. **Require real execution output**. A prediction is not a result, and generated tests are not evidence that anything ran.

DO NOT START UNTIL

At least one decision is genuinely blocked on an unknown. An experiment that cannot change a decision is a hobby.

COMPLETION TEST

Every high-impact question has either evidence behind a decision, or a written limitation with a workable fallback. The team has agreed which prototype code is kept and which is deleted.

MAKE IT CONCRETE FOR YOUR PRODUCT

Write down the one operational promise you are least sure you can keep — a latency, a delivery guarantee, a background behaviour on a locked device. Measure it this week, not after the build.

COMMON TRAP · Letting an experiment quietly become production code without reviewing its error handling, its secrets, and whether it can be deployed and supported.

01 DEFINE

02 DESIGN

03 VALID.

04 PLAN

05 ENABLE

06 BUILD

07 RELEASE

08 LEARN

04

THE DELIVERY LOOP · EVERY SLICE

Delivery planning

What is the smallest useful increment we can finish and verify? Planning is where the size of a change is decided — and size is the single biggest lever on risk.

ACCOUNTABLE OWNER

Product owner and engineering lead, together.

START WITH

Accepted decisions, validated assumptions, ranked scope, and the acceptance journeys from stage 01.

ACTIVITIES

- › **Slice vertically.** Each slice connects one user action to its permission check, its persistence, its events, and a result the user can see. A slice that ends at an API is not a slice.
- › Write **task briefs, not tickets**: requirement IDs, context, dependencies, contracts, acceptance criteria, failure cases, and the evidence expected at completion. Page 15 has the template.
- › Agree a definition of ready and a definition of done. Limit work in progress. Estimate from the scope you know, **not from how fast code can be generated**.

DELIVERABLES

A ranked backlog of vertical slices, milestone outcomes, task briefs, and a requirement-to-test traceability map (page 12).

WHERE AI HELPS · AND WHERE IT MUST NOT

Proposes decomposition and dependency order, flags an oversized change, drafts acceptance tests. **Humans choose priority, staffing, and how much change the team can actually review.** Review capacity, not typing speed, is the constraint.

DO NOT START UNTIL

The contracts a slice depends on are stable enough to build against, or the slice explicitly owns defining them.

COMPLETION TEST

The next three tasks each have an input, an owner, and an outcome somebody else could verify without asking the author what they meant.

MAKE IT CONCRETE FOR YOUR PRODUCT

If a slice cannot be finished, reviewed, tested and demonstrated inside two weeks, cut it until it can. The cut you resist is usually the one that was hiding an unresolved decision.

COMMON TRAP · A backlog organised by layer — all database work, then all API work, then all UI work. It delays every useful piece of feedback to the end of the project.

01 DEFINE

02 DESIGN

03 VALID.

04 PLAN

05 ENABLE

06 BUILD

07 RELEASE

08 LEARN

05

LAY THE GROUND · ONCE, THEN GROWN

Engineering foundation

Can anyone build, test, run and deploy the same system? Build only as much of this as the first slice needs. Then grow it one slice at a time.

ACCOUNTABLE OWNER

Engineering, with the platform or operations owner.

START WITH

The first slice's design, repository conventions, and the deployment target you actually intend to use.

ACTIVITIES

- › Version control, repository layout, build conventions, shared packages, and **repeatable container image builds**. No build may depend on an undeclared path on somebody's laptop.
- › **One documented command** from a clean checkout to a running system: migrations, seed data, and at least two tenants or fixtures. Secrets stay outside the repository.
- › Continuous integration for build, tests, contracts and dependency checks. Health and readiness endpoints, structured logs, a correlation ID, a staging deployment, and a rehearsed way back.

DELIVERABLES

A reproducible development environment, a CI pipeline, a deployment skeleton, shared libraries, and short repository instructions written **for people and for AI agents**.

WHERE AI HELPS · AND WHERE IT MUST NOT

Drafts scaffolding, container files, CI configuration and onboarding docs. **Run every generated command on the real environment** and check what is actually inside the image. A plausible pipeline file is not a working pipeline.

DO NOT START UNTIL

You know which components the first slice needs. Anything else you set up now is a guess you will maintain.

COMPLETION TEST

A person who has never seen the repository clones it, runs the documented commands, and reaches a working system. One correlation ID can then be followed across every component it touched.

MAKE IT CONCRETE FOR YOUR PRODUCT

The repository instructions are now a product input, not a courtesy. An AI agent given a vague repository produces vague changes; the fix is a precise, short, honest set of conventions.

COMMON TRAP · Building an internal platform before delivering a usable feature. Or believing a green local build proves the container image works.

01 DEFINE

02 DESIGN

03 VALID.

04 PLAN

05 ENABLE

06 BUILD

07 RELEASE

08 LEARN

06

THE DELIVERY LOOP · EVERY SLICE

Incremental implementation

Does this change do what the user needs, under realistic conditions? The stage AI changed the most — which is exactly why its review rules must be the strictest.

ACCOUNTABLE OWNER

The implementing engineer, and a named reviewer who is accountable for accepting it.

START WITH

A ready task brief, accepted contracts, test fixtures, and the current state of the repository.

ACTIVITIES

- › Implement **one behaviour across every layer it touches**. Keep authorisation, idempotency, acknowledgement and error handling with the feature that needs them, not in a later hardening task.
- › Test the **requirement**, not the implementation. Add integration or browser tests wherever a boundary is crossed or a visible state changes.
- › Review the diff, demonstrate the result, merge one coherent change, deploy to test. Update contracts and documentation **in the same change**.

DELIVERABLES

A working increment, a reviewed change, the tests that matter, deployment evidence, and updated documentation.

WHERE AI HELPS · AND WHERE IT MUST NOT

Implements bounded tasks, proposes tests, explains changes, assists review. Give it exact repository context and acceptance criteria. **A second AI review finds real issues. It does not replace an executed test or an accountable human reviewer.**

DO NOT START UNTIL

The task states its acceptance criteria and its failure cases. Without them, nobody can say later whether the change is finished.

COMPLETION TEST

Acceptance criteria pass, checks are green, **the reviewer can explain the change in their own words**, and the deployed behaviour can be demonstrated. No unexplained failing test.

MAKE IT CONCRETE FOR YOUR PRODUCT

Set a size a reviewer can genuinely hold in their head and treat it as a rule, not a hope. When a generated change exceeds it, split the task rather than skimming the diff.

COMMON TRAP · Accepting a large generated change because it looks complete. Or letting its tests repeat the same wrong assumption as the code they were generated beside.

01 DEFINE

02 DESIGN

03 VALID.

04 PLAN

05 ENABLE

06 BUILD

07 RELEASE

08 LEARN

07

THE DELIVERY LOOP · EVERY SLICE

Release validation

Can we put this in users' hands and support it? Two different questions, and the second one is the one teams answer last.

ACCOUNTABLE OWNER

A named release owner, with product, engineering and operations.

START WITH

An integrated release candidate, the acceptance journeys, the service targets, and a deployment plan.

ACTIVITIES

- › Run acceptance on the **supported devices and browsers**, not the ones the team happens to own. Exercise the complete journeys, the role restrictions, and every recovery path.
- › Validate isolation between tenants, behaviour under realistic load, and durable acknowledgement. **Rehearse** a restore, a restart, and a failed deployment — do not merely document them.
- › Prepare a limited rollout, monitoring, release notes and a runbook. Name who answers when it breaks, and how the release is turned off.

DELIVERABLES

A release evidence summary, a written list of known limitations, tested recovery and rollback procedures, release notes, and a rollout decision.

WHERE AI HELPS · AND WHERE IT MUST NOT

Generates test scenarios, analyses failures, drafts runbooks and release notes. Use sanitised logs and dedicated test data. **The release owner weighs the evidence and accepts the remaining limitations.**

DO NOT START UNTIL

The rollback path exists and someone has used it at least once, in a rehearsal, on this system.

COMPLETION TEST

The required journeys and service targets pass in the agreed environment; the remaining limitations are written down and accepted by name; deployment and recovery have been performed, not described.

MAKE IT CONCRETE FOR YOUR PRODUCT

Separate three things that get confused: the server **accepted** it, the connected client **received** it, and the person was **notified**. Measure each one on its own.

COMMON TRAP · Treating a green pipeline as proof of operational readiness. Or trusting a third-party delivery service for something that must never be missed.

01 DEFINE

02 DESIGN

03 VALID.

04 PLAN

05 ENABLE

06 BUILD

07 RELEASE

08 LEARN

08

ALWAYS ON · NEVER COMPLETED

Operate and improve

Is the product useful, reliable, and getting better? The only stage with no exit criterion, because leaving it means the product is over.

ACCOUNTABLE OWNER

Product and operations owners, supported by engineering.

START WITH

Real usage, service telemetry, user feedback, incident findings, and what the system costs to run.

ACTIVITIES

- › Track **outcomes, not activity**: completed journeys, successful onboarding, work that reached the person it was meant for. Pair each outcome with its latency, failure rate, lag and recovery time.
- › Investigate every incident, fix the cause, and turn each finding into **a test or a design change**. A finding that becomes neither will be found again.
- › Choose the next slice from observed value and observed cost. Update requirements and supersede decision records when evidence changes an answer — and keep the old record readable.

DELIVERABLES

A monitored service, incident learning that changed something, a refreshed backlog, and requirement and architecture records that match the system as it now is.

WHERE AI HELPS · AND WHERE IT MUST NOT

Summarises telemetry, groups feedback, suggests diagnostic hypotheses, drafts targeted fixes. **Check every explanation against the evidence**, and measure whether the change improved the user's outcome rather than the graph.

THE SIGNAL YOU ARE DRIFTING

Delivery speed rises while review time, rework, escaped defects and recovery time rise with it. That is not velocity; it is debt arriving early.

RHYTHM, NOT COMPLETION

Each improvement ends with a verified outcome and feeds the next planning cycle. The return path **08 → 01** is the one loop that runs for the product's whole life.

MAKE IT CONCRETE FOR YOUR PRODUCT

Choose no more than five numbers you will actually look at every week. A dashboard nobody reads is a cost, not an observability capability.

COMMON TRAP · Measuring AI by lines of code or closed tasks, while review effort, rework, defect escape and user experience all quietly get worse.

SCALE THE PROCESS TO THE TEAM

Cadence and right-sizing

A process that costs more than the risk it removes is waste. This page says which stages repeat, how often, and **what to drop when the team is small**. Dropping a stage is a decision; forgetting one is an accident.

CADENCE — HOW OFTEN EACH STAGE ACTUALLY RUNS

STAGE	RUNS	TRIGGERED AGAIN BY	TIMEBOX THAT KEEPS IT HONEST
01 Define	Once, then on change	A stage 08 finding, or a market change	Days, not months. Rank and move on.
02 Design	Once, then on change	A stage 03 result, a new boundary	Only as far as the first slice needs.
03 Validate	Per risky assumption	Any decision blocked on an unknown	Each experiment finishes in days.
04 Plan	Every slice	The previous slice completing	Hours per slice, not a planning week.
05 Enable	Once, then grown	A slice needing a capability it lacks	Grow it; never pause delivery to rebuild it.
06 Build	Every slice	A ready task brief	Two weeks maximum from brief to demo.
07 Release	Every release	An integrated candidate	Rehearsals are scheduled, not improvised.
08 Learn	Continuous	—	A fixed weekly review of five numbers.

RIGHT-SIZING — WHAT TO KEEP AT THREE TEAM SIZES

ARTEFACT	1–3 PEOPLE	4–12 PEOPLE	REGULATED OR SAFETY-CRITICAL
Requirements baseline	One file, IDs and acceptance examples	Versioned baseline, ranked	Versioned, signed, traced to controls
Architecture views	Two drawings	Three views plus sequences	Full set, reviewed and dated
Decision records	Short ADRs, 5–8 of them	ADRs per boundary decision	ADRs with an approval record
Validation	Top 3 risks only	Ranked risk list with owners	Evidence pack per control
Task briefs	Brief per slice	Brief per task	Brief plus traceability entry
Release evidence	A checklist plus a demo	Evidence summary and runbook	Formal acceptance record
Never drop, any size	An accountable owner per decision · an acceptance example per requirement · an executed test before merge · a rehearsed way back		

FAILURE MODES OF THE PROCESS ITSELF

- › **Gate theatre.** Stages become approvals to pass rather than questions to answer. The test: can anyone name a decision a gate actually changed?
- › **Documents nobody reads.** If a document has not been opened since it was written, it is not a baseline, it is a receipt. Shrink it.
- › **Front-loading.** Stages 01–05 expand to fill the time available because they feel safe. The cure is a dated first slice, agreed in advance.
- › **Evidence drift.** A number from six months ago on an unnamed machine is a rumour. Every figure carries its date and where it was measured.

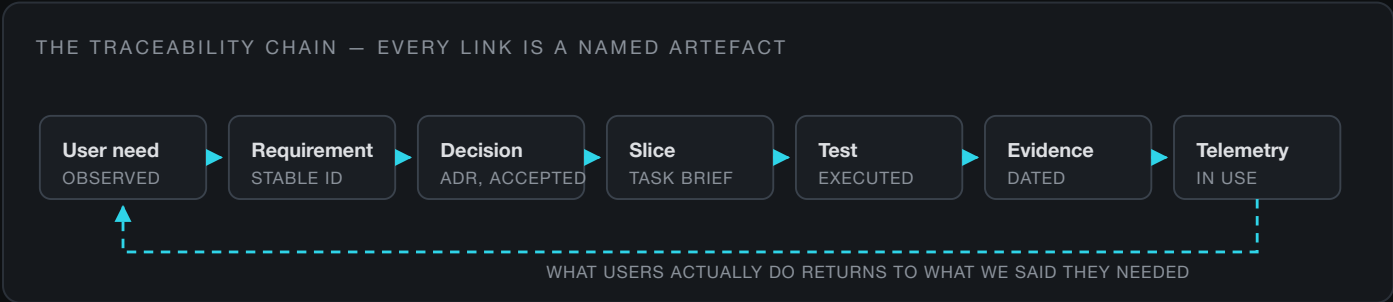
THE THREADS THAT RUN THROUGH EVERY STAGE

Cross-cutting concerns and traceability

Some concerns belong to no single stage, so they end up belonging to nobody. Each one below is **decided** in one stage, **proved** in another, and **watched** for ever. If a row has no name against it, that row is currently unowned.

DECIDED HERE · PROVED HERE · WATCHED HERE

CONCERN	DECIDED IN	PROVED IN	WATCHED IN	THE QUESTION THAT MUST HAVE AN ANSWER
Identity and access	02 Design	03 Validate	08 Learn	Who is allowed to do this, and which component enforces it?
Tenant / data isolation	02 Design	03 & 07	08 Learn	What stops one customer's data reaching another?
Personal data and privacy	01 Define	07 Release	08 Learn	What is collected, why, for how long, and who can erase it?
Security of the supply chain	05 Enable	06 Build	08 Learn	What is in the image, and who approved it being there?
Accessibility	02 Design	06 Build	07 Release	Can the journey be completed without a mouse, and with a screen reader?
Observability	05 Enable	06 Build	08 Learn	Can one user's request be followed end to end after the fact?
Resilience and recovery	02 Design	07 Release	08 Learn	What has actually been restored from a backup, and when?
Cost to run	02 Design	03 Validate	08 Learn	What does one more customer cost, and where does that curve bend?



THE CHAIN TEST

Pick one requirement ID. Walk it forward to the executed test and the dated evidence, and back to the user need.

If any link is missing, that requirement is not delivered — it is only implemented.

KEEP IT CHEAP

The chain is a habit, not a tool purchase. A requirement ID in the branch name, in the test name and in the decision record gets you most of it for free.

TEMPLATE · STAGE 02 DELIVERABLE

The architecture decision pack

Enough design to make the first slice concrete and reviewable. Most products need **five or six decisions** before they can build anything honestly. Resolve only those; give every other open question an owner and a date.

WHAT TO PRODUCE

One system-context drawing; one deployable-component drawing; a sequence for sign-in, for the main command, and for whatever is delivered asynchronously; a data-ownership matrix; the module or service contracts; and a first-slice deployment view.

THE DECISIONS ALMOST EVERY PRODUCT HAS TO MAKE

DECISION	WHAT IT MUST SETTLE
ADR-001 · Identity and membership	Which identity provider, hosted or custom sign-in, how an organisation maps to a customer, and what happens to an active session when membership is removed.
ADR-002 · Tenant isolation	The isolation boundary and how it is enforced in each store, queue and cache. Provisioning, credential scope, cross-tenant access, and the cost and scaling assumption behind the choice.
ADR-003 · Data authority and history	One authoritative write path per aggregate. Whether history is a stored fact or a derived convenience. Concurrency control, replay, retention, and any outbox.
ADR-004 · Module and service boundaries	What is one deployable unit and what is not. Versioned interfaces between them, shared packages, what happens when one is unavailable, and the compatibility rule for deploying them separately.
ADR-005 · Deployment and operations	Local topology, staging, secrets handling, telemetry, durable storage configuration, backup and the recovery approach.
ADR-006 · Non-functional targets	The numbers the product promises: latency, availability, durability, and what the product does when it cannot meet them. A target with no stated failure behaviour is a wish.

A DECISION RECORD IS SEVEN FIELDS

- Context — the forces, in the team's own words.
- Options — at least two that were genuinely considered.
- Decision — one sentence, active voice.
- Consequences — what gets harder, not only what gets better.
- Evidence — what was measured, when, on what.
- Owner — a person, not a team.
- Status — proposed, accepted, or superseded by a later record.

KEEP THE STATUS VISIBLE AND THE HISTORY INTACT

A proposed decision becomes accepted only after review. A later decision **supersedes** an earlier one rather than editing it away, so the reasoning stays readable a year later. ^[1]

Number records globally and never renumber them. A renumbered decision breaks every link that ever pointed at it.

WHAT A DECISION PACK IS NOT

It is not a design of the whole system, and it is not an approval to stop thinking. Its only job is to make the first slice buildable by someone who was not in the room.

TEMPLATE · MILESTONE ONE

The first working slice

One thin path through every layer, proved end to end, **before** the full first release is attempted. It is an integration proof, not a feature. Its purpose is to establish the pattern every later slice copies.

THE DEMONSTRATION JOURNEY — FILL IN YOUR OWN NOUNS

Sign in → **application shell loads** → **choose the authorised context (customer, tenant, workspace)** → **submit one real command** → **durable acknowledgement** → **a second user sees the result live**.

FIVE STEPS, EACH ONE PROVABLE ON ITS OWN

STEP	WHAT IT PROVES
1 • Identity and context	A user signs in, their memberships load, they select a context they are allowed, and a context they are not allowed is refused — including when the identifier is altered by hand.
2 • Shell and modules	Versioned modules load, the shared theme applies, the active context is visible, and an unavailable module produces a recoverable error rather than a blank screen.
3 • Command and durability	One command is submitted with an idempotency key. Permission and state transition are validated, the write reaches the authoritative store, and only then is it acknowledged.
4 • Event and projection	The durable event is published and consumed, a read model updates, and the change reaches an authorised observer. A syncing or unavailable state is shown honestly.
5 • Evidence	Repeat the command. Restart the consumer. Attempt cross-tenant access. Switch context. Sign out. Confirm no duplicate, no leak, and no stale subscription.

SCOPE DISCIPLINE

Two synthetic tenants, two scoped identities, one observer session. Load only the modules the journey needs, and implement only the controls it uses. **Declare the authoritative write path before building it**, not after.

MILESTONE ACCEPTANCE

A clean checkout runs the whole journey from documented commands. CI builds the images and passes the relevant tests. **A reviewer repeats the demonstration unaided** and follows one correlation ID through every component.

WHAT THIS MILESTONE DOES AND DOES NOT ESTABLISH

It establishes an **implementation pattern**: how a command is authorised, made durable, observed and demonstrated. It does not establish feature completeness. The remaining requirements follow as further slices, each copying this shape.

SIGNALS THE SLICE IS TOO BIG

- › It cannot be demonstrated until several people have merged work in a particular order.
- › Its acceptance criteria contain the word "and" more than three times.
- › Nobody can say which decision record it depends on.

WORKING AGREEMENT

Making AI-assisted work reviewable

AI does not remove work; it moves it. Writing gets cheaper, so **context, review and evidence become the constraint**. This page is the agreement that keeps that constraint visible.

THE LOOP

Brief the task → **inspect the real context** → **propose a small change** → **implement** → **execute the checks** → **review the diff and the behaviour** → **deploy and observe**. Return to the brief whenever evidence changes an assumption. Never skip "execute the checks" because the change looks obviously right; that is precisely when it is skipped.

A USEFUL TASK BRIEF — SIX FIELDS

- › **Outcome** — one user-visible behaviour.
- › **Traceability** — requirement IDs and accepted decision records.
- › **Context** — the exact files, contracts, fixtures and conventions.
- › **Boundaries** — what is in scope, and what existing behaviour must not change.
- › **Acceptance** — success, plus the realistic failure cases.
- › **Evidence** — commands to run, results to report, limitations to declare.

WHAT AI MUST NEVER OWN

- › **Priority**. What matters is a human judgement about users and money.
- › **Acceptance**. A person accepts a requirement, a decision and a release.
- › **Evidence**. A claim that a test passes is not a test passing.
- › **Accountability**. Every merged change has a human name against it.
- › **Secrets and real personal data**. They stay out of prompts and fixtures unless the environment is explicitly approved.

WORKED EXAMPLE OF A BRIEF — ADAPT THE NOUNS

"Implement the **submit-order** command and its control, for requirement **ORD-01** and decision **ADR-003**. The server derives the customer from verified membership, never from the request body. A repeated idempotency key returns the same result. Two concurrent submissions respect the single-active rule. Another customer cannot observe the event. Demonstrate with two tenants and **report the actual test output**."

WHO REVIEWS WHAT

- › **Product owner** — scope and acceptance.
- › **Technical lead** — architecture and contracts.
- › **Engineer** — implementation and verification.
- › **Reviewer** — challenges the assumption, not the formatting.
- › **Release owner** — readiness and recovery.
- › In a small team one person holds several roles — but never both author and sole acceptor of the same risky change.

RUNNING AI WORK IN PARALLEL

- › Only where contracts and file ownership are already clear.
- › Give **integration one owner**, always.
- › Keep each change small and reconcile before adding another stream.
- › Concurrency multiplies review load. Add it when review capacity exists, not when the queue looks empty.

MEASURE THE SYSTEM, NOT THE OUTPUT

Track **review and rework time, escaped defects, change failure rate, recovery time** and real user outcomes alongside delivery speed. Small batches and fast feedback are what turn AI adoption into better delivery, rather than into more work in progress. [2, 3]

ADOPT IT · ADAPT IT · KEEP IT PROPORTIONAL

Using this guide

Take it as a working guide informed by established engineering practice, not as a claim that every organisation follows this exact sequence. Keep the process proportional to the size of the product and the cost of being wrong.

THREE ACTIONS THAT START IT

- › **Design.** Produce the decision pack on page 13. Resolve only the decisions the first slice needs; give every other open question an owner and a date.
- › **Validate.** Run experiments against your three riskiest assumptions. Record the results with their date and machine, and update the proposed decisions.
- › **Deliver.** Turn the page 14 journey into a small backlog, build the minimum foundation it needs, and complete the first repeatable demonstration.

GLOSSARY

- › **ADR** — architecture decision record: one decision, its options and its consequences.
- › **Vertical slice** — one user action delivered through every layer it touches.
- › **Acceptance example** — a concrete case showing a requirement is met.
- › **Definition of ready / done** — the agreed entry and exit tests for a task.
- › **Idempotency key** — a value that makes a repeated request safe.
- › **Correlation ID** — one identifier followed across every component of a request.
- › **Service target** — a promised number, with the behaviour when it is missed.

WHERE THE ARTEFACTS LIVE

- › [docs/requirements/](#) — the product baseline and acceptance journeys.
- › [docs/architecture/](#) — drawings, contracts and decision records.
- › [docs/development/](#) — this playbook, delivery planning, task briefs.
- › [docs/operations/](#) — runbooks, release notes, incident findings.
- › Adjust the paths to your repository. What matters is that each artefact has **one** home and that it is discoverable without asking.

DOCUMENT STATUS

Version 1.0. This guide proposes a development process. It does not approve any architecture decision and does not indicate that any implementation, experiment or acceptance test has been completed. Adapt the stage names to the language your team already uses; the stages matter, the labels do not.

REFERENCES

- › **[1] AWS · Architectural decision record process**
docs.aws.amazon.com/prescriptive-guidance/latest/architectural-decision-records/adr-process.html
 Supports the decision-record structure, ownership and lifecycle used on page 13.
- › **[2] DORA · Working in small batches**
dora.dev/capabilities/working-in-small-batches/
 Supports small increments, fast feedback and bounded AI-assisted changes.
- › **[3] DORA · State of AI-assisted Software Development 2025**
dora.dev/research/2025/dora-report/
 Research context for integrating AI with the surrounding engineering system.

References checked 17 September 2026. The stage model and templates are a synthesis for practical use, not a prescribed universal standard.